Pricing an option by solving a PDE

Serial implementation, optimisation and benchmarking

Zain Al-Saffi, 48008361

COSC3500 Milestone 1

August 2026

What an option is

The right, but not the obligation, to buy at a fixed price

- A European call is the right, but not the obligation, to buy a share for a fixed **strike** price on a fixed day.
- At expiry its value is known exactly. Above the strike it pays the difference, below it pays nothing, so the payoff has a kink at the strike.
- The problem is the price **today**, ninety-one days before expiry.

The payoff at expiry, with the kink at the strike.

5200

today's share price

5250

the strike

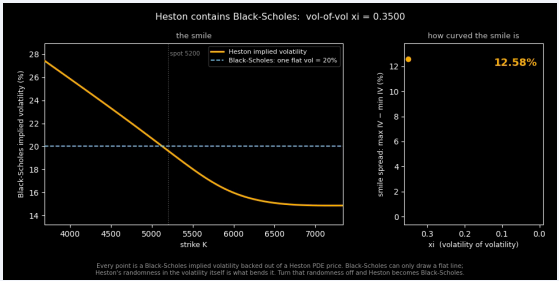
91 days

to expiry

Why not Black-Scholes

Implied volatility is not constant across strikes

- Black-Scholes assumes the share's volatility is **one constant number**.
- Implied volatility computed from market option prices varies across strikes, which contradicts that assumption.
- The fix is a model in which the volatility itself varies.



Implied volatility from Heston PDE prices. The dashed line is the single Black-Scholes volatility.

- Setting Heston's volatility of volatility to zero recovers Black-Scholes exactly. That is the main validation test.

The share price and its variance are both stochastic processes

$$dS = (r - q)\, S\, dt + \sqrt{v}\, S\, dW_1$$
$$dv = \kappa(\theta - v)\, dt + \xi\sqrt{v}\, dW_2, \qquad \operatorname{corr}(dW_1, dW_2) = \rho$$

- S is the share price and v is its instantaneous variance, which follows its own stochastic process.
- κ is the rate at which the variance reverts to its long-run level θ, and ξ sets the size of the variance fluctuations.
- The two noises are correlated at $\rho = -0.70$.

- The correlation is negative because volatility tends to rise when prices fall.

The symbols in the two equations

Definitions

Symbol	Definition
S	the share price
v	the variance of the share price, the square of its volatility
r	the risk-free interest rate
q	the dividend yield of the share
dt	a small step forward in time
$dW_1,\ dW_2$	random increments, one for the price and one for the variance
κ	the rate at which the variance reverts to its long-run level
θ	the long-run level of the variance
ξ	the volatility of volatility, which sets the size of variance fluctuations
ρ	the correlation between the two increments

From stochastic model to deterministic equation

There is no random number generator in the program

- The fair price is an average. It is the expected payoff over every path the market could take, discounted back to today.
- One way to compute an average is to simulate many random paths and take their mean. This program does not do that.
- The Feynman-Kac theorem says that average, viewed as a function of the starting point, solves a deterministic equation. That function is the option price, written V. The following slides construct its equation.

- Averaging over the noise leaves a deterministic equation, so the program needs no random numbers.

The derivative notation

Subscripts denote partial derivatives

- $V(S, v, t)$ is the option price when the share price is S, the variance is v and the time is t. It is the unknown the program solves for.
- Capital V and lowercase v are different quantities. V is the option price, v is the variance of the share price.
- A subscript denotes a partial derivative, taken with the other variables held fixed.

Symbol	Definition
V_S	the first derivative with respect to the share price
V_{SS}	the second derivative with respect to the share price
V_v	the first derivative with respect to the variance
V_{vv}	the second derivative with respect to the variance
V_{Sv}	the mixed derivative, in the share price and the variance

- On the grid each of these becomes a difference between neighbouring cells, which is why

Why the noise terms become second derivatives

Averaging over one time step

- Over one small time step the noise moves the share price up or down by about $\sqrt{v}\,S$, with both directions equally likely.
- Averaging the value over the up move and the down move cancels the first-derivative contributions.
- The remaining term is second order, half the move size squared times V_{SS}.

move size $\sqrt{v}\,S$ so the noise in S becomes $\frac{1}{2}\,(\sqrt{v}\,S)^2\,V_{SS} \;=\; \frac{1}{2}\,vS^2\,V_{SS}$

- The same argument applied to the variance noise, of size $\xi\sqrt{v}$, gives $\frac{1}{2}\,\xi^2 v\,V_{vv}$. The correlation between the two noises gives the mixed term.

The equation, term by term

Each part of the model contributes one term

Part of the model	**Resulting term**
time passing	V_t
the drift of S, which is $(r-q)\,S$	$(r-q)\,S\,V_S$
the drift of v, which is $\kappa(\theta - v)$	$\kappa(\theta - v)\,V_v$
the noise in S, of size $\sqrt{v}\,S$	$\frac{1}{2}\,vS^2\,V_{SS}$
the noise in v, of size $\xi\sqrt{v}$	$\frac{1}{2}\,\xi^2 v\,V_{vv}$
the correlation ρ between the noises	$\rho\,\xi\,vS\,V_{Sv}$
discounting at the rate r	$r\,V$

- Drift terms give first derivatives and noise terms give second derivatives. The drift terms need no averaging because they are not random.

The equation the program solves

Every quantity except V is known

The Heston pricing equation

$$V_t + (r-q)\,S\,V_S + \kappa(\theta - v)\,V_v + \tfrac{1}{2} v S^2\,V_{SS} + \rho\,\xi\,v S\,V_{Sv} + \tfrac{1}{2}\xi^2 v\,V_{vv} = r\,V$$

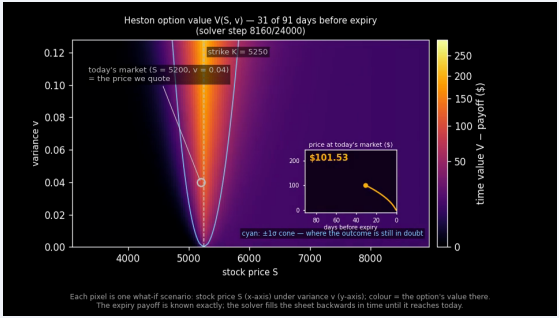
- Every symbol except V is a known number before the solve starts. r and q come from the market, and κ, θ, ξ, ρ and the starting variance are the model's five parameters. The strike sits in the expiry payoff, not in the equation.
- The unknown is the option price $V(S, v, t)$, a whole function with one value per scenario, and the grid is that function written out cell by cell.

- The reported price is V evaluated at today's share price and variance.

The solver

A grid over share price and variance, solved backwards from expiry

- Share price runs across the grid and variance runs down it. Each cell holds the option price V for that pair of inputs.
- The **spacing** is the gap between neighbouring values, about $10 between share prices here. Finer spacing tracks the true curve better and multiplies the number of cells.
- At expiry every cell equals the payoff, so the solver starts there and steps **backwards** to today.
- Each step rebuilds every cell from its nine neighbours, reading one buffer and

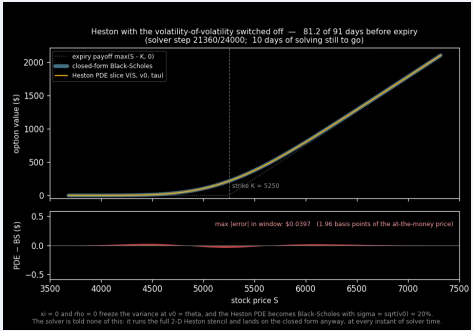


Mid-solve. Colour is the option's value above the payoff.

Validation

Two properties the solver must have, run as automated tests

- With $\xi = 0$ and $\rho = 0$, Heston reduces to Black-Scholes, which has a closed form. The solver runs the full stencil unchanged and must reproduce the closed form. Measured error 1.16e-3.
- Put-call parity is a model-free identity between call and put prices. The residual is 5.4e-6 and is unchanged while the price varies by thirty dollars across parameter settings.



The $\xi = 0$ run matching closed-form Black-Scholes at every step of the solve.

Three independent methods

Grid, Fourier and Monte Carlo share no code

- Two independent methods, sharing no code with the grid solver, price the same contract.

Method	Price
this solver, on the grid	\$196.1684
Fourier inversion, semi-analytic	\$196.1692
Monte Carlo, four hundred thousand paths	\$196.13 $\pm$ \$0.55

- The grid and Fourier prices agree to **four parts in a million**, and the Monte Carlo interval brackets both.

How the optimisation was structured

Seven solvers, one technique per level

- Every level is a full copy of the level below with exactly **one transformation** added, so a difference between neighbours measures one technique.
- Every level must match the baseline's answer before it is benchmarked.

- A single before and after gives one ratio without attribution. The ladder attributes the speedup to individual techniques.

Level 0, harness control

Adds no technique

baseline — the reference stencil src/solver_baseline.cpp:80–89

```
// finite differences declarations (what we use to calc the pde)
double V_S = (east - west) / (2.0 * g.stock_spacing());
double V_SS = (east - 2.0 * V + west) / (g.stock_spacing() * g.stock_spacing());
double V_v = (north - south) / (2.0 * g.variance_spacing());
double V_vv = (north - 2.0 * V + south) / (g.variance_spacing() * g.variance_spacing());
double V_Sv = (ne - nw - se + sw) / (4.0 * g.stock_spacing() * g.variance_spacing());
// heston pde update, its the model the simulation is based on
double pde = V + dt * (0.5 * v * S * S * V_SS + rho * xi * v * S * V_Sv + 0.5 * xi * xi * v * V_vv
    + (rate - div_yield) * S * V_S + kappa * (theta - v) * V_v - rate * V);
next[g.index(stock_i, var_j)] = pde;
```

level 0 — the same arithmetic, line for line src/solver_opt_kernels.cpp:54–67

```
double V_S = (east - west) / (2.0 * g.stock_spacing());
double V_SS = (east - 2.0 * V + west) /
              (g.stock_spacing() * g.stock_spacing());
double V_v = (north - south) / (2.0 * g.variance_spacing());
double V_vv = (north - 2.0 * V + south) /
              (g.variance_spacing() * g.variance_spacing());
double V_Sv = (ne - nw - se + sw) /
              (4.0 * g.stock_spacing() * g.variance_spacing());
double pde = V + dt * (0.5 * v * S * S * V_SS +
                       rho * xi * v * S * V_Sv +
                       0.5 * xi * xi * v * V_vv +
                       (rate - div_yield) * S * V_S +
                       kappa * (theta - v) * V_v - rate * V);
next[g.index(stock_i, var_j)] = pde;
```

No techniques applied. Measured: −0.1 % (Table 2), so the ladder harness itself costs nothing and changes no answers.

- This level runs the baseline's exact arithmetic through the ladder's kernel selection, changing nothing else.
- If selecting a kernel through a function pointer cost anything, every rung above would be measuring the harness rather than its technique.

- The baseline's arithmetic, line for line, behind the ladder's dispatch. Throughput was unchanged, so later differences measure the techniques themselves.

Level 1, hoisting

Loop-invariant computation moved outside the loops

Level 0 — everything recomputed per cell src/solver_opt_kernels.cpp:41-61

```
    for (int var_i = 1; var_i < g.num_variance_nodes() - 1; ++var_i) {
        for (int stock_i = 1; stock_i < g.num_stock_nodes() - 1; ++stock_i) {
            double S = g.stock_price(stock_i);
            double V = cur[g.index(stock_i, var_i)];
            double v = g.variance(var_i);
            double east = cur[g.index(stock_i + 1, var_i)];
            double west = cur[g.index(stock_i - 1, var_i)];
            double north = cur[g.index(stock_i, var_i + 1)];
            double south = cur[g.index(stock_i, var_i - 1)];
            double ne = cur[g.index(stock_i + 1, var_i + 1)];
            double nw = cur[g.index(stock_i - 1, var_i + 1)];
            double se = cur[g.index(stock_i + 1, var_i - 1)];
            double sw = cur[g.index(stock_i - 1, var_i - 1)];
            double V_S = (east - west) / (2.0 * g.stock_spacing());
            double V_SS = (east - 2.0 * V + west) /
                          (g.stock_spacing() * g.stock_spacing());
            double V_v = (north - south) / (2.0 * g.variance_spacing());
            double V_vv = (north - 2.0 * V + south) /
                          (g.variance_spacing() * g.variance_spacing());
            double V_Sv = (ne - nw - se + sw) /
                          (4.0 * g.stock_spacing() * g.variance_spacing());
```

level 1 — hoisted per step and per row src/solver_opt_kernels.cpp:114-121, 126-129, 135-139

```
    const double two_ds = 2.0 * stock_spacing;
    const double ds_sq = stock_spacing * stock_spacing;
    const double two_dv = 2.0 * variance_spacing;
    const double dv_sq = variance_spacing * variance_spacing;
    const double four_ds_dv = 4.0 * stock_spacing * variance_spacing;
    // The cost of carry and the v=0 drift are constants of the market.
    const double carry = rate - div_yield;
    const double kappa_theta = kappa * theta;
    // ...
    std::vector<double> stock(static_cast<std::size_t>(num_stock_nodes));
    for (int stock_i = 0; stock_i < num_stock_nodes; ++stock_i) {
        stock[stock_i] = stock_i * stock_spacing;
    }
    // ...
        const double v = var_j * variance_spacing;
        const double half_v = 0.5 * v;
        const double rho_xi_v = (rho * xi) * v;
        const double half_xi2_v = (0.5 * xi) * xi) * v;
        const double mean_rev = kappa * (theta - v);
```

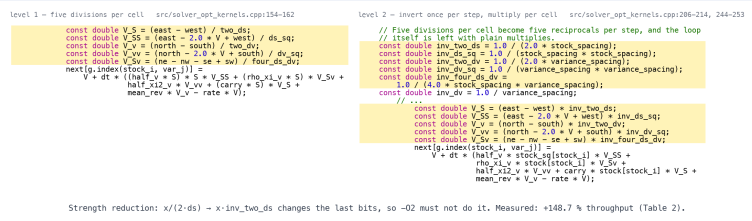
Hoisting, a lookup table and CSE move work to the level where it varies. Names only, never reordering, so the answer stays bit-identical. Measured: +0.3 % (Table 2).

- Values that do not change inside a loop, such as grid spacings, their reciprocals and per-row coefficients, are computed once outside it and given names.
- The intended win is fewer repeated operations per cell, if the compiler has not already removed them.

- No measurable change. GCC at -O2 had already hoisted all of this, and this level demonstrates that.

Level 2, strength reduction

A transformation the compiler is not permitted to make



- Each division by a grid constant becomes a multiplication by its reciprocal, computed once before the loops.
- The inner loop had five divisions per cell, and division is the slowest arithmetic the core does.

- $x/(2\,ds)$ and $x \cdot (1/(2\,ds))$ round differently in the last bits, so -O2 must leave the divisions alone. A divide costs 5 to 14 cycles and does not pipeline, a multiply costs about one.

Levels 3 and 4

The baseline already did both, so throughput did not change

level 2 — g.index arithmetic on every access src/solver_opt_kernels.cpp:227–243

```
for (int var_i = 1; var_i < num_variance_nodes - 1; ++var_i) {
    const double v = var_i * variance_spacing;
    const double half_v = 0.5 * v;
    const double rho_xi_v = (rho * xi) * v;
    const double half_xi2_v = ((0.5 * xi) * xi) * v;
    const double mean_rev = kappa * (theta - v);

    for (int stock_i = 1; stock_i < num_stock_nodes - 1; ++stock_i) {
        const double V = curr[g.index(stock_i, var_i)];
        const double west = curr[g.index(stock_i - 1, var_i)];
        const double east = curr[g.index(stock_i + 1, var_i)];
        const double north = curr[g.index(stock_i, var_i - 1)];
        const double south = curr[g.index(stock_i, var_i + 1)];
        const double ne = curr[g.index(stock_i + 1, var_i + 1)];
        const double nw = curr[g.index(stock_i - 1, var_i + 1)];
        const double se = curr[g.index(stock_i + 1, var_i - 1)];
        const double sw = curr[g.index(stock_i - 1, var_i - 1)];
```

level 3 — row bases, memory walked in layout order src/solver_opt_kernels.cpp:316–337

```
for (int var_i = 1; var_i < num_variance_nodes - 1; ++var_i) {
    // The three rows the stencil reads, each addressed once per row.
    const int row = var_i * num_stock_nodes;
    const int row_above = row + num_stock_nodes;
    const int row_below = row - num_stock_nodes;

    const double v = var_i * variance_spacing;
    const double half_v = 0.5 * v;
    const double rho_xi_v = (rho * xi) * v;
    const double half_xi2_v = ((0.5 * xi) * xi) * v;
    const double mean_rev = kappa * (theta - v);

    for (int stock_i = 1; stock_i < num_stock_nodes - 1; ++stock_i) {
        const double V = curr[row + stock_i];
        const double west = curr[row + stock_i - 1];
        const double east = curr[row + stock_i + 1];
        const double north = curr[row_above + stock_i];
        const double south = curr[row_below + stock_i];
        const double ne = curr[row_above + stock_i + 1];
        const double nw = curr[row_above + stock_i - 1];
        const double se = curr[row_below + stock_i + 1];
        const double sw = curr[row_below + stock_i - 1];
```

Variance outside, stock inside, three row bases per row. Measured: +0.3 % (Table 2). The baseline already walked this way, and the ctl-order control prices the wrong way.

Level 3, traversal order. The loops walk each row in the order it sits in memory, so every 64-byte cache line is fully used before it is evicted.

level 3 — bounds tested inside the loop condition src/solver_opt_kernels.cpp:316–328

```
for (int var_i = 1; var_i < num_variance_nodes - 1; ++var_i) {
    // The three rows the stencil reads, each addressed once per row.
    const int row = var_i * num_stock_nodes;
    const int row_above = row + num_stock_nodes;
    const int row_below = row - num_stock_nodes;

    const double v = var_i * variance_spacing;
    const double half_v = 0.5 * v;
    const double rho_xi_v = (rho * xi) * v;
    const double half_xi2_v = ((0.5 * xi) * xi) * v;
    const double mean_rev = kappa * (theta - v);

    for (int stock_i = 1; stock_i < num_stock_nodes - 1; ++stock_i) {
```

level 4 — plain locals, one loop per equation src/solver_opt_kernels.cpp:407–416, 447–456

```
    // The bounds are worked out once, so each loop condition is a plain
    // integer comparison and neither body contains a conditional.
    const int last_stock = num_stock_nodes - 1;
    const int last_var = num_variance_nodes - 1;

    // Every cell in the interior follows the full PDE with no special cases.
    for (int var_i = 1; var_i < last_var; ++var_i) {
        const int row = var_i * num_stock_nodes;
        const int row_above = row + num_stock_nodes;
        const int row_below = row - num_stock_nodes;
        // ...
    // The transport row is peeled out into its own loop rather than being
    // tested on every cell of the interior.
    const int row_one = num_stock_nodes;
    for (int stock_i = 1; stock_i < last_stock; ++stock_i) {
        const double V = curr[stock_i];
        const double V_3 = (curr[stock_i + 1] - curr[stock_i - 1]) * inv_two_ds;
        const double V_v = (curr[row_one + stock_i] - curr[stock_i]) * inv_dv;
        next[stock_i] = V + dt * (carry + stock[stock_i] * V_3
                                  kappa_theta + V_v - rate * V);
    }
```

Loop splitting and a branch-free interior. No cell tests which equation it follows. Measured: −0.2 % (Table 2), and the ctl-branch control times the fused version.

Level 4, loop splitting. The $v = 0$ row gets its own loop instead of an if in every cell.

- Throughput did not change at either level, because the baseline already walked rows in memory order and already split the $v = 0$ row out. What the walking order is worth, the deliberately swapped version on the ladder slide shows.

Level 5, induction variables

Row pointers instead of index arithmetic

level 4 — row + stock_i on every access src/solver_opt_kernels.cpp:413–433

```
for (int var_j = 1; var_j < last_var; ++var_j) {
    const int row = var_j * num_stock_nodes;
    const int row_above = row + num_stock_nodes;
    const int row_below = row - num_stock_nodes;

    const double v = var_j * variance_spacing;
    const double half_v = 0.5 * v;
    const double rho_xi_v = (rho * xi) + v;
    const double half_xi2_v = (0.5 * xi) * xi) * v;
    const double mean_rev = kappa * (theta - v);

    for (int stock_i = 1; stock_i < last_stock; ++stock_i) {
        const double V = cur[row + stock_i];
        const double east = cur[row + stock_i + 1];
        const double west = cur[row + stock_i - 1];
        const double north = cur[row_above + stock_i];
        const double south = cur[row_below + stock_i];
        const double ne = cur[row_above + stock_i + 1];
        const double nw = cur[row_above + stock_i - 1];
        const double se = cur[row_below + stock_i + 1];
        const double sw = cur[row_below + stock_i - 1];
```

level 5 — raw row pointers, addressing does the work src/solver_opt_kernels.cpp:507–530

```
for (int var_j = 1; var_j < last_var; ++var_j) {
    // The row bases move on by exactly one row per iteration, which turns
    // the per-cell index arithmetic into plain pointer indexing.
    const double* const row_mid = cur_base + var_j * num_stock_nodes;
    const double* const row_above = row_mid + num_stock_nodes;
    const double* const row_below = row_mid - num_stock_nodes;
    double* const out_row = next_base + var_j * num_stock_nodes;

    const double v = var_j * variance_spacing;
    const double half_v = 0.5 * v;
    const double rho_xi_v = (rho * xi) + v;
    const double half_xi2_v = (0.5 * xi) * xi) * v;
    const double mean_rev = kappa * (theta - v);

    for (int stock_i = 1; stock_i < last_stock; ++stock_i) {
        const double V = row_mid[stock_i];
        const double east = row_mid[stock_i + 1];
        const double west = row_mid[stock_i - 1];
        const double north = row_above[stock_i];
        const double south = row_below[stock_i];
        const double ne = row_above[stock_i + 1];
        const double nw = row_above[stock_i - 1];
        const double se = row_below[stock_i + 1];
        const double sw = row_below[stock_i - 1];
```

Induction variable simplification. Ten index additions per cell become pointer indexing. Measured: +2.2 % (Table 2).

- Each row is walked with pointers that advance one step per cell, instead of recomputing row times width plus column for every neighbour.
- The intended win is about ten fewer integer additions per cell, if that work is not already hidden.

- A small gain, the only one after level 2. Neighbour offsets are computed inside the load instructions at no extra cost, and address adds run on separate execution ports, so most of this work was already hidden behind the floating point.

Level 6, unrolling

Four cells per loop iteration

level 5 — one cell per iteration src/solver_opt_kernels.cpp:521–541

```
        for (int stock_i = 1; stock_i < last_stock; ++stock_i) {
            const double V = row_mid[stock_i];
            const double east = row_mid[stock_i + 1];
            const double west = row_mid[stock_i - 1];
            const double north = row_above[stock_i];
            const double south = row_below[stock_i];
            const double ne = row_above[stock_i + 1];
            const double nw = row_above[stock_i - 1];
            const double se = row_below[stock_i + 1];
            const double sw = row_below[stock_i - 1];
            const double V_S = (east - west) * inv_two_ds;
            const double V_SS = (east - 2.0 * V + west) * inv_ds_sq;
            const double V_v = (north - south) * inv_two_dv;
            const double V_vv = (north - 2.0 * V + south) * inv_dv_sq;
            const double V_Sv = (ne - nw - se + sw) * inv_four_ds_dv;
            out_row[stock_i] =
                V + dt * (half_v * stock_sq[stock_i] * V_SS +
                          rho_xi_v * stock[stock_i] * V_Sv +
                          half_xi2_v * V_vv + carry * stock[stock_i] * V_S +
                          mean_rev * V_v - rate * V);
        }
```

level 6 — four independent cells per iteration src/solver_opt_kernels.cpp:617–627, 707–710

```
        int stock_i = 1;
        // Four independent cells per loop iteration. Each block is level 5's
        // body copied out with its index and local names numbered, and the
        // only real difference is that four of them now sit between one pair
        // of loop-back branches instead of one.
        for (; stock_i + 3 < last_stock; stock_i += 4) {
            const int i0 = stock_i;
            const int i1 = stock_i + 1;
            const int i2 = stock_i + 2;
            const int i3 = stock_i + 3;

            // ...
        }
        // Whatever is left over when the row is not a multiple of four.
        for (; stock_i < last_stock; ++stock_i) {
            const double V = row_mid[stock_i];
```

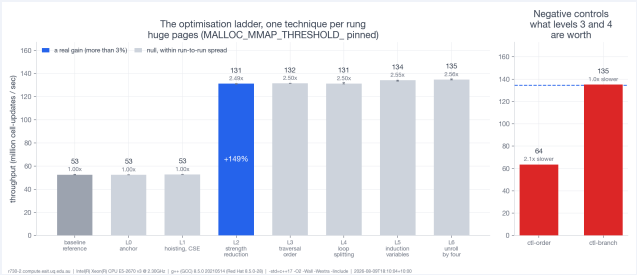
Unroll by four. Four dependency chains between one pair of loop-back branches. Measured: +0.5 % (Table 2). At -O2 the compiler already unrolls counted loops.

- The inner loop handles four cells per iteration instead of one.
- That means one loop test per four cells, and more independent arithmetic in flight to hide instruction latencies.

- No measurable gain. The compiler already unrolls counted loops at -O2, so manual unrolling duplicates its work.

The ladder, measured

Only level 2 changed throughput



52.6
baseline Mcell per s

131.2
after strength reduction

134.8
level 6

2.56x
overall speedup

- Swapping the loops on purpose makes identical arithmetic **2.1x slower**, so the traversal order the baseline already had was doing real work. An automated test holds every level to the baseline answer, within 15 units in the last decimal place.

The benchmark protocol

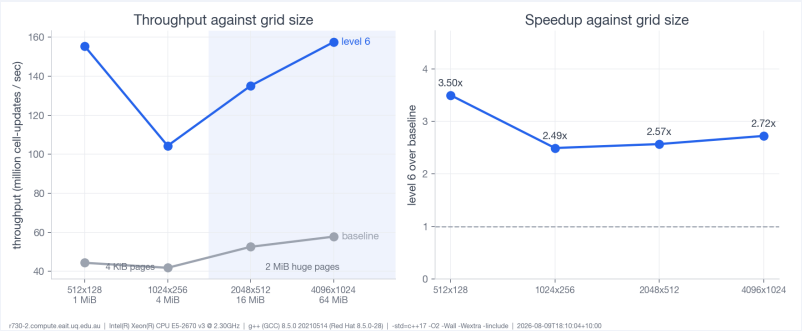
How every number in this talk was produced

Step	Rule
where jobs run	a reserved rangpur compute node, r730-2, via sbatch
before every sweep	the binary rebuilds and the full test suite must re-pass
repetitions	five per point, the median is reported
provenance	every CSV stamps host, job id, compiler, flags, CPU, caches
the allocator	pinned, so the page regime is a choice, not an accident

- The plot scripts reject any CSV without its provenance header, so no laptop measurement can enter a figure.

Scaling with problem size

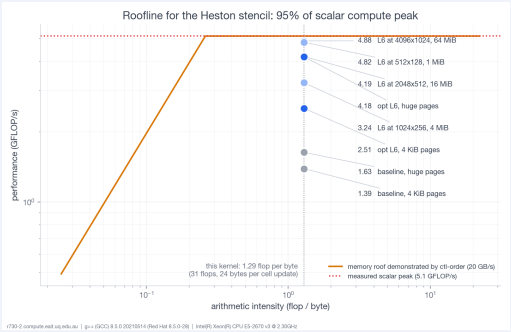
One to sixty-four mebibytes of working set



- The first run of this sweep confounded page size with cache. This is the re-run with the allocator pinned, and the speedup holds at 2.5x to 3.5x across grids.

The roofline

Testing the memory-bound assumption



- The solver was assumed to be memory-bound. It is not. Optimisation moved it from division-bound to compute-bound, with level 6 at **82 percent** of the measured scalar peak.

The page-size effect

A 1.66x change from memory pages alone

Solver	4 KiB pages	2 MiB pages	From pages alone
baseline	44.8	52.6	1.17x
level 2	80.0	131.2	1.64x
level 6	**81.1**	**134.8**	**1.66x**

- Million cell updates per second. Same binary, same node, and not a code change at all.
- Sixteen mebibytes is eight huge pages or four thousand small ones, against a translation buffer with a thousand entries.

- It was found because the median of five repetitions was averaging two different memory regimes.

Reflection

Four questions the spec asks

The main conclusion

At -O2 the compiler has already done the textbook transformations, so the hand optimisations that pay are the ones it is not allowed to do.

The most surprising result

Page size changed throughput more than five of the six code techniques combined, without any code change.

What I would do differently

Reserve the node exclusively from the first job. An early shared-node measurement was real, but it answered a different question.

Milestone 2

The kernel is now compute-bound, so SIMD first, then OpenMP across rows. The two-buffer design already makes every cell independent within a step.

A 2.56x serial speedup, validated at every level

COSC3500 Milestone 1, Zain Al-Saffi

Where the strike price enters

The model and the contract are separate

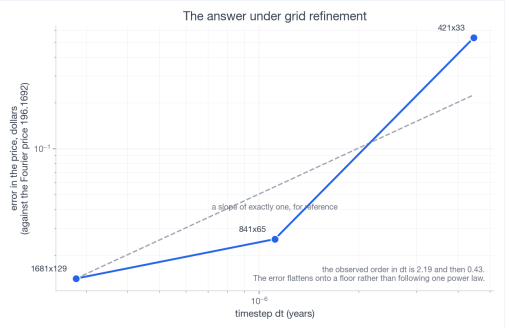
- The two equations describe how the share price moves. They do not involve the option contract, which is why no strike appears in them.
- The contract enters at expiry, where the grid is filled with the payoff, $\max(S - K, 0)$ for this call. That is where the strike K enters.
- Black-Scholes merges the model and the contract into one closed formula, which is why S and K appear together there. The PDE route keeps them separate.

- Heston does not build on the Black-Scholes formula. It relaxes one Black-Scholes assumption, the constant volatility. With $\xi = 0$ the model reduces to Black-Scholes, which is the main validation test.

Convergence

Halve the grid spacing and the error should fall four times over

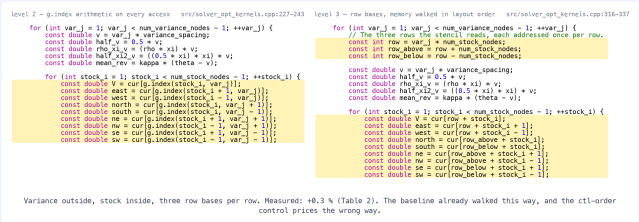
- The same option is priced on a sequence of grids, each with half the spacing of the last, and each price is compared with the Fourier reference.
- The error falls about **four times over per halving**. That is the second-order accuracy the stencil is built to have, so the slope is itself a check on the code.
- Below about \$0.014 refinement stops helping. The grid has to end somewhere, and the reference carries its own small error, so those two set the floor.



Error against the Fourier price as the spacing halves.

Level 3, traversal order

The inner loop follows the memory layout

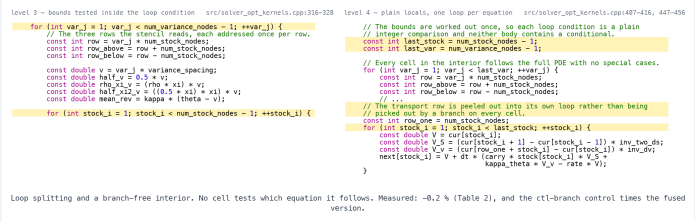


- The loop nest is arranged so the inner loop visits cells in the exact order they sit in memory.
- Sequential access lets the cache and prefetcher stream whole lines, while a strided walk wastes most of every line it fetches.

- No change, by construction. The baseline already traverses memory in layout order.

Level 4, loop splitting

A separate loop for the boundary row



- The $v = 0$ boundary row gets its own loop, instead of an if test on every cell of the main loop.
- A branch in every cell can mispredict and stall the pipeline, so splitting keeps the main loop free of branches.

- No change, by construction. The baseline already splits the boundary row into its own loop, and the fused version is one of the negative controls.

What the speedup cost

Fifteen units in the last place of the answer

```
./test_opt_matches (local run)

opt L0          call: max |diff| 0.000e+00   (bit-identical required)  OK
opt L1          call: max |diff| 0.000e+00   (bit-identical required)  OK
opt L2          call: max rel  3.296e-15   (tol 1e-11)             OK
opt L3          call: max rel  3.296e-15   (tol 1e-11)             OK
opt L4          call: max rel  3.296e-15   (tol 1e-11)             OK
opt L5          call: max rel  3.296e-15   (tol 1e-11)             OK
opt L6          call: max rel  3.296e-15   (tol 1e-11)             OK
opt ctl-order   call: max rel  3.296e-15   (tol 1e-11)             OK
opt ctl-branch call: max rel  3.296e-15   (tol 1e-11)             OK
opt L0          put : max |diff| 0.000e+00   (bit-identical required)  OK
opt L1          put : max |diff| 0.000e+00   (bit-identical required)  OK
opt L2          put : max rel  0.000e+00   (tol 1e-11)             OK
opt L3          put : max rel  0.000e+00   (tol 1e-11)             OK
opt L4          put : max rel  0.000e+00   (tol 1e-11)             OK
opt L5          put : max rel  0.000e+00   (tol 1e-11)             OK
opt L6          put : max rel  0.000e+00   (tol 1e-11)             OK
opt ctl-order   put : max rel  0.000e+00   (tol 1e-11)             OK
opt ctl-branch put : max rel  0.000e+00   (tol 1e-11)             OK
test_opt_matches: PASS (9 of 9 level(s) written)


    Every rung and both controls, held to the baseline's answer.
```

- The whole speedup cost 3.3e-15 relative error, and an automated gate holds every level and both controls to it.

The full ladder

Sweep E, huge pages, median of five

Level	Adds	Mcell per s	Step gain
baseline	the reference solver	52.6	
level 0	nothing, the harness control	52.6	$-0.1\,\%$
level 1	hoisting, tables, CSE	52.7	$+0.3\,\%$
level 2	**strength reduction**	**131.2**	$\mathbf{+148.7\,\%}$
level 3	traversal order	131.5	$+0.3\,\%$
level 4	loop splitting	131.3	$-0.2\,\%$
level 5	induction variables	134.2	$+2.2\,\%$
level 6	unroll by four	134.8	$+0.5\,\%$
ctl-order	loops swapped on purpose	63.6	2.1x slower
ctl-branch	boundary branch fused back	135.3	free

The negative controls

Each control removes a property the baseline already had

level 5 — variance outside, stock inside src/solver_opt_kernels.cpp:507–521

```
    for (int var_j = 1; var_j < last_var; ++var_j) {
        // The row bases move on by exactly one row per iteration, which turns
        // the per-cell index arithmetic into plain pointer indexing.
        const double* const row_mid = cur_base + var_j * num_stock_nodes;
        const double* const row_above = row_mid + num_stock_nodes;
        const double* const row_below = row_mid - num_stock_nodes;
        double* const out_row = next_base + var_j * num_stock_nodes;

        const double v = var_j * variance_spacing;
        const double half_v = 0.5 * v;
        const double rho_xi_v = (rho * xi) * v;
        const double half_xi2_v = ((0.5 * xi) * xi) * v;
        const double mean_rev = kappa * (theta - v);

        for (int stock_i = 1; stock_i < last_stock; ++stock_i) {
```

ctl-order — swapped on purpose src/solver_opt_kernels.cpp:806–822

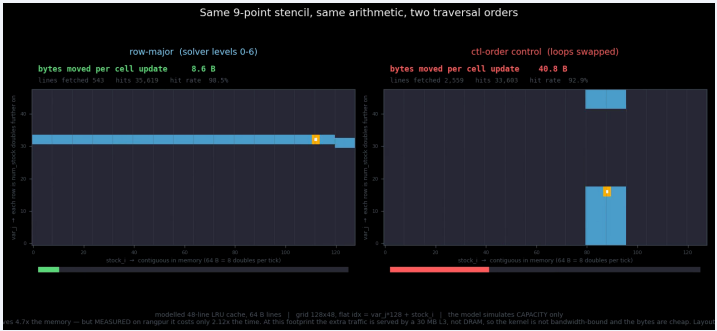
```
    // The order here is wrong on purpose. The row constants can no longer be
    // hoisted either, because they now change on every inner iteration, and
    // that is the second cost of getting the traversal backwards.
    for (int stock_i = 1; stock_i < last_stock; ++stock_i) {
        for (int var_j = 1; var_j < last_var; ++var_j) {
            const double* const row_mid = cur_base + var_j * num_stock_nodes;
            const double* const row_above = row_mid + num_stock_nodes;
            const double* const row_below = row_mid - num_stock_nodes;
            double* const out_row = next_base + var_j * num_stock_nodes;

            const double v = var_j * variance_spacing;
            const double half_v = 0.5 * v;
            const double rho_xi_v = (rho * xi) * v;
            const double half_xi2_v = ((0.5 * xi) * xi) * v;
            const double mean_rev = kappa * (theta - v);

            const double V = row_mid[stock_i];
```

Same arithmetic, same tables, loops swapped: 2.1× slower (Table 3). This is what the null level-3 rung was worth.

- Swapping the loops makes the inner iteration jump a whole row per step. Same arithmetic, **2.1x slower**. That measures the value of the baseline's traversal order.
- Fusing the boundary row back into the interior loop has no measurable cost. The branch predictor handles the per-cell branch.

Modelled cache traffic

Why the swapped loop order is slower



A modelled LRU cache under both traversal orders. Row-major moves 8.6 bytes per cell update, the swapped order moves 40.8.